

# Google Sheets Integration Playbook

## 1. Overview

This playbook documents the repeatable steps we used to connect the Kerry Williams Realty site to Google Sheets via Apps Script + a lightweight webhook. You can reuse the same pattern for any future client portal, intake form, or CRM sync.

## 2. Prerequisites

- Google Workspace account with access to Google Sheets and Apps Script.
- A sheet that will act as the system of record (create a blank one per client).
- Column plan for the tabs you need (e.g., `Contact Leads`, `Agents`, `Properties`).
- An environment that can host JSON or static assets (GitHub Pages, Netlify, etc.).
- Basic familiarity with JavaScript `fetch()`, Git, and deploying static sites.

## 3. Sheet Structure

1. Create the workbook and rename Tab 1 to `Contact Leads`.
2. Add the headers we expect (`Timestamp`, `Name`, `Email`, `Phone`, `Saved Homes Count`, etc.). Keep naming consistent because the script appends columns by index.
3. Add additional tabs (`Agents`, `Properties`) with columns matching the JSON schema you want to expose publicly.
4. Freeze row 1 in each tab to lock the headers.

## 4. Apps Script Web App

1. In the Sheet, click **Extensions → Apps Script**.
2. Replace the default `Code.gs` with the `update-refresh.gs` file from this repo.
3. Update `MASTER_SHEET_ID` to the ID of the sheet you just created.
4. Deploy the script as a Web App:
  - **Deploy → Manage deployments → New deployment**
  - Type: Web app
  - Execute as: Me
  - Who has access: Anyone
  - Copy the Web App URL; this is your webhook endpoint.
5. Whenever you change the script, redeploy so the new version is live.

## 5. Static JSON Export (optional)

- The `exportData()` + `saveJsonToDrive()` helper regenerates `properties-1.json`. Run `doPost?action=refresh` (or run `handleRefreshPost`) to upload the JSON to Drive, then sync it to your site.
- Use this when you need a static snapshot for anonymous browsing, while still logging private events (contacts, saved homes) through the webhook.

## 6. Front-End Wiring

1. Add `sheet-config.js` to your site and set `window.SheetSettings.accountWebhookUrl` to the Web App URL.
2. Use the lightweight `SheetSync.sendAccount(payload)` helper (see `sheet-sync.js`). It packages the payload, sends it as a simple POST (no preflight), and surfaces success/failure states.

3. In the create-account modal (or any form), collect the fields you want logged and call ``SheetSync.sendAccount`` after saving locally.
4. Provide user feedback (e.g., “Saved locally + logged to the Google Sheet”) so the workflow is transparent.

## 7. CORS & Reliability Checklist

- Keep the Apps Script in sync with the deployed version; redeploy after every change.
- Ensure the fetch uses a simple ``Content-Type`` (e.g., ``text/plain``) to avoid OPTIONS preflights.
- Gracefully handle ``fetch`` failures by storing the data locally and retrying when the webhook is reachable.
- Log both success and error events in the console for quick debugging.

## 8. Reuse Playbook for New Businesses

1. Duplicate the Google Sheet (File → Make a copy) and rename it for the new client.
2. Update ``MASTER_SHEET_ID`` and redeploy the Apps Script to create a fresh webhook URL.
3. Configure the new site's ``sheet-config.js`` to point to that URL.
4. Tailor the JSON schema or contact headers if the new business needs extra columns.
5. Document the endpoints + schema in an internal Notion page for each client so onboarding is repeatable.

## 9. Security Notes

- Web App access is set to “Anyone”; only include non-sensitive data in the payload.
- For sensitive workflows, require a signed token or restrict to Google accounts you control.
- Periodically prune old data in the sheet or move it into BigQuery / a database if volume grows.

## 10. Operational Tips

- Add a refresh trigger in Apps Script (time-driven) to regenerate JSON nightly.
- Use Sheet version history or backups to guard against accidental edits.
- Automate QA by submitting a test lead weekly to confirm the pipeline works.